

Task Manager V1 Design Document

Project Purpose

The purpose of this project is not simply to create a task manager.

The primary goal is to improve:

- Java Programming
- Object-Oriented Programming (OOP)
- Software Engineering
- Problem Solving
- Project Planning
- Debugging
- Refactoring
- Git Usage

The task manager is the vehicle used to practice these skills.

Project Requirements

The user must be able to:

1. Add Task
2. View Tasks
3. Mark Task Complete
4. Delete Task
5. Exit Program

Version 1 limitations:

- Tasks only exist while the program is running.
 - Tasks are not saved to a file.
 - Duplicate task names are allowed.
 - No priorities.
 - No due dates.
 - No categories.
-

System Architecture

User ↓ Main ↓ TaskManager ↓ Task

Information flows downward through method calls and upward through returned information.

Core Design Rule

When deciding where a method belongs, ask:

1. What data does the method need?
2. What data does the method change?
3. Who owns that data?
4. Put the method there.

Examples:

markComplete() → Task

deleteTask() → TaskManager

showMenu() → Main

UML Diagram

```
+-----+ | Main | +-----+ | - scanner | | - taskManager | +-----+ | + run() | | +  
showMenu() | | + handleChoice() | | + getTaskName() | | + getTaskNumber() | +-----+ | | v  
  
+-----+ | TaskManager | +-----+ | - tasks | +-----+ | + addTask() | | + deleteTask() | |  
+ completeTask() | | + getTaskList() | | + getTaskByNumber() | | + hasTasks() | +-----+ | | v  
  
+-----+ | Task | +-----+ | - name | | - completed | +-----+ | + getName() | | +  
isCompleted() | | + markComplete() | | + markIncomplete() | | + changeName() | | + toString() |  
+-----+
```

Task Class

Purpose

Represents a single task.

Responsibilities

- Store task name

- Store completion status
- Rename itself
- Mark itself complete
- Mark itself incomplete
- Represent itself as text

Fields

String name

boolean completed

Methods

Task(name)

getName()

isCompleted()

markComplete()

markIncomplete()

changeName()

toString()

Pseudocode

Task(name): set name set completed to false

getName(): return name

isCompleted(): return completed

markComplete(): set completed to true

markIncomplete(): set completed to false

changeName(newName): set name to newName

toString(): if completed return name + " [X]" else return name + " []"

Rules

- New tasks start incomplete.
 - Task owns its own data.
 - Other classes should not directly modify Task fields.
-

TaskManager Class

Purpose

Manages the collection of tasks.

Responsibilities

- Add tasks
- Delete tasks
- Find tasks
- Complete tasks
- Build task list displays
- Validate task numbers

Fields

`ArrayList<Task> tasks`

Methods

`addTask(name)`

`deleteTask(taskNumber)`

`completeTask(taskNumber)`

`getTaskByNumber(taskNumber)`

`getTaskList()`

`hasTasks()`

Pseudocode

`addTask(name):` create Task add Task to tasks

hasTasks(): return whether tasks list contains any tasks

getTaskByNumber(taskNumber): convert task number to index return task at index

completeTask(taskNumber): get task using getTaskByNumber() tell task to mark itself complete

deleteTask(taskNumber): get task using getTaskByNumber() remove task from tasks

getTaskList(): create empty result

```
loop through tasks
  add task number
  add task string representation

return result
```

Rules

- TaskManager owns the collection.
 - TaskManager validates task numbers.
 - TaskManager coordinates actions.
 - TaskManager should not handle user input.
-

Main Class

Purpose

Handles user interaction and program flow.

Responsibilities

- Display menu
- Read user input
- Display messages
- Call TaskManager methods

Fields

Scanner scanner

TaskManager taskManager

boolean programRunning

Methods

run()

showMenu()

handleChoice(choice)

getTaskName()

getTaskNumber()

Pseudocode

run(): set programRunning to true

```
while programRunning
  showMenu()
  get user choice
  handleChoice(choice)
```

showMenu(): display "1. Add Task" display "2. View Tasks" display "3. Complete Task" display "4. Delete Task" display "5. Exit Program"

getTaskName(): display "Enter task name:" read input return task name

getTaskNumber(): display "Enter task number:" read input return task number

handleChoice(choice):

```
if choice == 1
  get task name
  tell TaskManager to add task
  display success message

if choice == 2
  get task list from TaskManager
  display task list

if choice == 3
  get task number
  tell TaskManager to complete task
```

```
        display success/failure message

if choice == 4
    get task number
    tell TaskManager to delete task
    display success/failure message

if choice == 5
    set programRunning to false

otherwise
    display invalid choice message
```

Rules

- Main handles all user interaction.
 - Main should not manipulate task collections directly.
 - Main delegates work to TaskManager.
-

Build Order

Phase 1

- Create project
- Create classes

Phase 2

- Implement Task

Phase 3

- Implement TaskManager

Phase 4

- Implement Main

Phase 5

- Connect all features

Phase 6

- Test all features

Phase 7

- Refactor
-

Git Milestones

Commit 1

Create project structure

Commit 2

Implement Task constructor

Commit 3

Implement getters

Commit 4

Implement complete/incomplete methods

Commit 5

Implement rename and toString

Commit 6

Implement TaskManager

Commit 7

Implement Main menu

Commit 8

Connect features

Commit 9

Testing and cleanup

Testing Checklist

Task Tests

Constructor

Expected:

- Name set correctly
- Completed = false

getName()

Expected:

- Returns task name

isCompleted()

Expected:

- Returns current completion status

markComplete()

Expected:

- Completed becomes true

markIncomplete()

Expected:

- Completed becomes false

changeName()

Expected:

- Name changes

toString()

Expected:

Incomplete: Study Java []

TaskManager Tests

addTask()

Expected:

- Task appears in list

hasTasks()

Expected:

- False when empty
- True when tasks exist

getTaskByNumber()

Expected:

- Correct task returned

completeTask()

Expected:

- Task becomes complete

deleteTask()

Expected:

- Task removed from list

getTaskList()

Expected:

- Numbered task list returned

Main Tests

Add Task

Expected:

- Task added
- Return to menu

View Tasks

Expected:

- Tasks displayed

Complete Task

Expected:

- Task marked complete

Delete Task

Expected:

- Task removed

Exit

Expected:

- Program closes
-

Edge Cases

View tasks when empty

Expected:

- "No tasks found"

Complete invalid task number

Expected:

- Error message
- Program continues running

Delete invalid task number

Expected:

- Error message
- Program continues running

Invalid menu option

Expected:

- Error message
- Return to menu

Major Design Decisions

1. Java over Python because it aligns with coursework and OOP goals.
 2. Console application first to focus on programming fundamentals.
 3. Task contains only:
 4. name
 5. completed
 6. Task numbering starts at 1 for users.
 7. TaskManager owns the collection of tasks.
 8. Task owns its own state.
 9. Main handles all user interaction.
 10. Use helper methods to avoid duplicated logic.
 11. Build bottom-up: Task → TaskManager → Main
 12. Follow: Requirements → Design → Pseudocode → Code → Testing → Refactoring
-

Future Versions

Version 2

- Save tasks to file
- Load tasks from file

Version 3

- Rename tasks from menu
- Search tasks
- Filter completed tasks

Version 4

- Priority levels
- Due dates
- Categories

Version 5

- GUI/Desktop Application

These features are intentionally postponed until Version 1 is complete.